

Shahjalal University of Science and Technology
Department of Computer Science & Engineering



Course Title: Algorithm Design and Analysis

Course Code: CSE 0613 2137

Assignment

Experimental Performance Analysis of Quicksort Pivot Selection
Strategies

Submitted By:

Name: Md Adib Abrar
Reg. No.: 2024331032

Submitted To:

Md. Shadmim Hasan Sifat
Lecturer
Department of Computer Science and
Engineering.

Submitted on 14th July, 2026

Source Code Repository

The complete source code, implementation files, benchmark programs, and generated results for this project are available at the following GitHub repository:

<https://github.com/GreatTurnip/quicksort-pivot-benchmark>

N.B.: For a complete understanding of the implementation and experimental process, please visit the GitHub repository. It contains the full source code, project structure, and additional files that could not be included in this report.

1.QUICKSORT

Quicksort is a divide-and-conquer sorting algorithm. It works by selecting a pivot element from an array and partition the remaining elements in two groups. First one is the set of all the elements smaller than the pivot and the set of all the elements larger than the pivot. The pivot is then placed in its correct sorted position. Then, the same process is recursively applied to both the left and right subarrays until the entire array is sorted. Quicksort is efficient as it has the average time complexity of $O(n \log n)$. Although, its performance heavily depend on the choice of pivot. A poor pivot selection can result in unbalanced partition and increase its complexity upto the worst-case of $O(n^2)$. Therefore, it can be assessed that different pivot selection strategies; for example – first element, last element, middle element, and random element can significantly affect the algorithm's performance.

2.METHODOLOGY

This project compares how different pivot selection strategies affect Quicksort's efficiency. I implemented four common methods—first, last, middle, and random element—and benchmarked them using various array structures. The goal was to see how execution times fluctuate depending on the pivot choice and the initial state of the data.

2.1.Pivot Selection Strategies

The performance of the Quicksort algorithm is significantly influenced by the method used to select the pivot element. In this experiment, four different pivot selection strategies were implemented and evaluated: first element selection, last element selection, middle element selection, and random element selection.

First Element Selection:

In this approach, the first element of the array is selected as the pivot. Although this method is simple and requires minimal computation, it can result in poor performance when the input array is already sorted or nearly sorted. In such cases, the partitions become highly unbalanced, causing the algorithm to approach its worst-case time complexity of $O(n^2)$.

Last Element Selection:

In this strategy, the last element of the array is chosen as the pivot. It has similar characteristics to first element selection and can suffer from the same performance issues. For already sorted or reverse-sorted arrays, it often produces highly skewed partitions, leading to increased execution time.

Middle Element Selection:

In this method, the middle element of the array is selected as the pivot. This strategy generally results in more balanced partitions, allowing Quicksort to divide the problem more efficiently. As a result, it provides more stable performance across different types of input data.

Random Element Selection:

In this approach, the pivot is selected randomly from the array. By avoiding a fixed selection pattern, this strategy reduces the likelihood of consistently choosing an unfavorable pivot. Therefore, it provides more consistent performance and performs well across various input arrangements.

2.2.Test Cases

To evaluate the performance of different pivot selection strategies, the Quicksort algorithm was tested using three different types of input arrays. Each test case represents a different arrangement of data and helps analyze how input order affects the performance of the algorithm.

Random Array:

In this test case, the elements of the array are arranged in a random order. This represents a typical data distribution and helps evaluate the average performance of different pivot selection strategies.

Ascending Array:

In this test case, the elements are arranged in increasing order. This input type is useful for analyzing the behavior of pivot strategies that may perform poorly on already sorted data, such as first element and last element selection.

Descending Array:

In this test case, the elements are arranged in decreasing order. Similar to the ascending case, it helps observe the effect of highly ordered input on different pivot selection strategies and their ability to maintain efficient partitioning

2.3.Performance Measurement

To compare the efficiency of different Quicksort pivot selection strategies, the execution time of each algorithm was measured for every test case. Each pivot strategy was applied to arrays of different arrangements, including random, ascending, and descending orders. The time required to complete the sorting process was recorded and used as the primary performance metric.

The collected execution times were then analyzed and visualized using graphs to observe the differences between the pivot selection strategies. This analysis helps identify how the choice of pivot affects Quicksort's performance under different input conditions.

3.Implementation

The Quicksort benchmark was implemented using the C++ programming language. The program was organized into separate modules, where each pivot selection strategy was implemented independently to allow direct comparison between different approaches.

Each Quicksort implementation uses a partitioning function to rearrange elements around the selected pivot. The pivot selection method differs between implementations, while the overall Quicksort algorithm remains the same. The benchmark program generates input arrays of different arrangements, applies each pivot strategy, and measures the execution time required for sorting.

The execution results were collected and used to generate graphs for analyzing the performance differences between the pivot selection strategies under various input conditions.

3.1.Program Structure

The program was structured into multiple components to ensure a clear separation between the different Quicksort implementations and the benchmarking process. Each pivot selection strategy was implemented in a separate header file, allowing individual testing and comparison of their performance.

Here is the project structure:

```
quicksort-pivot-benchmark/
|
|— graphs/
|   |— graph_1_random_array.png
|   |— graph_2_ascending_array.png
|   └─ graph_3_descending_array.png
|
|— include/
|   |— QuickSortFirstElementPivot.h
|   |— QuickSortLastElementPivot.h
|   |— QuickSortMiddleElementPivot.h
|   |— QuickSortRandomElementPivot.h
|   └─ timer.h
|
|— Makefile
|
|— report/
|   └─ report.pdf
|
|— results/
|   |— ascending_result.csv
|   |— descending_result.csv
|   └─ random_result.csv
|
└─ src/
    |— graph_generator.py
    └─ main.cpp
```

3.1.1 Quicksort Implementations

Separate header files were created for each pivot selection strategy: first element pivot, last element pivot, middle element pivot, and random element pivot. Each header file contains the corresponding partition function and recursive Quicksort function implementation. This modular structure allows each pivot strategy to be tested independently and makes the comparison between different approaches more organized.

First Pivot Header File:

```
#pragma once

#include <vector>
#include <utility>

namespace FirstPivotQuickSort
{
    inline int partition(std::vector<int> &arr, int low, int high)
    {
        int pivot = arr[low];

        int i = low - 1;
        int j = high + 1;

        while (true)
        {
            do
            {
                i++;
            } while (arr[i] < pivot);

            do
            {
                j--;
            } while (arr[j] > pivot);

            if (i < j)
            {
                std::swap(arr[i], arr[j]);
            }
        }

        std::swap(arr[low], arr[j]);
        return j;
    }

    void quicksort(std::vector<int> &arr, int low, int high)
    {
        if (low < high)
        {
            int pivotIndex = partition(arr, low, high);
            quicksort(arr, low, pivotIndex - 1);
            quicksort(arr, pivotIndex + 1, high);
        }
    }
}
```

```

        } while (arr[j] > pivot);

        if (i >= j)
            return j;

        std::swap(arr[i], arr[j]);
    }
}

inline void quicksort(std::vector<int> &arr, int low , int high)
{
    if(low < high)
    {
        int pivotIndex = partition(arr, low, high);

        quicksort(arr, low, pivotIndex);
        quicksort(arr, pivotIndex+1, high);
    }
}
}

```

Last Pivot Header File:

```

#pragma once

#include <vector>
#include <utility>

namespace LastPivotQuickSort
{
    inline int partition(std::vector<int> &arr, int low, int high)

```

```
{  
    std::swap(arr[low], arr[high]);  
  
    int pivot = arr[low];  
    int i = low - 1;  
    int j = high + 1;  
  
    while (true)  
    {  
        do  
        {  
            i++;  
        } while (arr[i] < pivot);  
  
        do  
        {  
            j--;  
        } while (arr[j] > pivot);  
  
        if (i >= j)  
            return j;  
  
        std::swap(arr[i], arr[j]);  
    }  
}  
  
inline void quicksort(std::vector<int> &arr, int low, int high)  
{  
    if(low < high)
```

```

    {
        int pivotIndex = partition(arr, low, high);

        quicksort(arr, low, pivotIndex);
        quicksort(arr, pivotIndex+1, high);
    }
}
}

```

Middle Pivot Header File:

```

#pragma once

#include <vector>
#include <utility>

namespace MiddlePivotQuickSort
{
    inline int partition(std::vector <int> & arr, int low, int high)
    {
        int midIndex = low + (high - low) / 2;
        std::swap(arr[low], arr[midIndex]);
        int pivot = arr[low];

        int i = low - 1;
        int j = high + 1;

        while (true)
        {
            do{
                i++;
            }while (arr[i] < pivot);
            do

```

```

        {
            j--;
        } while (arr[j] > pivot);
        if( i >= j)
            return j;

        std::swap(arr[i], arr[j]);
    }

}

inline void quicksort(std::vector <int> &arr, int low, int high)
{
    if(low < high)
    {
        int p = partition(arr, low, high);
        quicksort(arr, low, p);
        quicksort(arr, p + 1, high);
    }
}
}

```

Random Pivot Header File:

```

#pragma once
#include <vector>
#include <cstdlib>
#include <ctime>
#include <utility>

```

```

namespace RandomPivotQuickSort
{
    inline int partition(std::vector<int> & arr, int low, int high)
    {
        int randomIndex = low + rand() % (high - low + 1);
        std::swap(arr[low], arr[randomIndex]);
        int pivot = arr[low];

        int i = low - 1;
        int j = high + 1;

        while (true)
        {
            do { i++; } while (arr[i] < pivot);
            do { j--; } while (arr[j] > pivot);

            if (i >= j)
                return j;

            std::swap(arr[i], arr[j]);
        }
    }

    inline void quicksort(std::vector<int> &arr, int low, int high)
    {
        if (low < high)
        {
            int p = partition(arr, low, high);
            quicksort(arr, low, p);
            quicksort(arr, p + 1, high);
        }
    }
}

```

3.1.2.Benchmark Driver

The benchmark driver is responsible for running the experiments, measuring execution time, and collecting the performance data of different Quicksort implementations. It consists of two main components: a timer module and the main benchmark program.

Timer Module (timer.h):

A separate header file was created to handle time measurement functionality. The timer function is used to accurately calculate the execution time required by each Quicksort implementation during the benchmark process. This allows the performance of different pivot selection strategies to be compared based on their sorting time.

```
#pragma once
#include <chrono>

template <typename Func>

double executionTimeCounter(Func f)
{
    auto start = std::chrono::high_resolution_clock::now();
    f();
    auto stop = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::micro> elapsedTime = stop - start;

    return elapsedTime.count();
}
```

Main Program (main.cpp):

The main program controls the overall benchmarking process. It generates different types of input arrays, including random, ascending, and descending arrays, and executes each Quicksort implementation on these inputs. The measured execution times are then stored in CSV files, which are later used for result analysis and graph generation.

```
#include <iostream>
#include <vector>
#include <fstream>
#include <random>
#include <ctime>
#include <cstdlib>

// my own headers

#include "../include/QuickSortFirstElementPivot.h"
#include "../include/QuickSortLastElementPivot.h"
#include "../include/QuickSortMiddleElementPivot.h"
#include "../include/QuickSortRandomElementPivot.h"
#include "../include/timer.h"

using namespace std;

int MIN_SIZE = 10;
int MAX_SIZE = 5000;
int STEP = 10;
int REPETITIONS = 20;

//random array generator

vector<int> randomArray(int n)
{
    vector<int> arr(n);
    for(int i = 0; i < n; i++) arr[i] = rand() % 1'000'000 + 1;
    return arr;
}

//generate ascending array
```

```

vector<int> ascendingArray(int n)
{
    vector<int> arr(n);
    for (int i = 0; i < n; i++) arr[i] = i;
    return arr;
}

//generate descending array

vector<int> descendingArray(int n)
{
    vector<int> arr(n);
    for(int i = 0; i < n; i++) arr[i] = n - i;
    return arr;
}

//run experiment function

void experiment(const string &arrayType, vector<int> (*generator)
(int), const string &ouputFile)
{
    ofstream out(ouputFile);
    out << "size,First,Last,Middle,Random\n";

    cout << "RUNNING EXPERIMENT :" <<arrayType <<endl;

    for(int n = MIN_SIZE; n <= MAX_SIZE; n += STEP)
    {
        double totalFirst = 0, totalLast = 0, totalMiddle = 0,
totalRandom = 0;
        for(int rep = 0; rep < REPETIOTIONS; rep++)
        {
            vector<int> base = generator(n); //this is the main copy

```

```

        vector<int> a = base;
        vector<int> b = base;
        vector<int> c = base;
        vector<int> d = base;
        totalFirst += executionTimeCounter(
            [&]()
            {
                FirstPivotQuickSort::quicksort(a, 0, (int)a.size()
- 1);
            }
        );
        totalLast += executionTimeCounter(
            [&]()
            {
                LastPivotQuickSort::quicksort(b, 0, (int)b.size()
- 1);
            }
        );
        totalMiddle += executionTimeCounter(
            [&]()
            {
                MiddlePivotQuickSort::quicksort(c, 0,
(int)c.size() - 1);
            }
        );
        totalRandom += executionTimeCounter(
            [&]()
            {
                RandomPivotQuickSort::quicksort(d, 0,
(int)d.size() - 1);
            }
        );
    }

```

```

        out << n << ", "
        << (totalFirst / REPETIOTIONS) << ", "
        << (totalLast / REPETIOTIONS) << ", "
        << (totalMiddle / REPETIOTIONS) << ", "
        << (totalRandom / REPETIOTIONS) << endl;

        if(n % 1000 == 0)    cout << " SIZE " << n << " DONE" << endl;
    }

    out.close();
    cout << "FINISHED AND SAVED AS: " << ouputFile << endl;
}

int main()
{
    srand(time(0));

    experiment("Random", randomArray, "results/random_result.csv");
    experiment("Ascending", ascendingArray,
"results/ascending_result.csv");
    experiment("Descending", descendingArray,
"results/descending_result.csv");

    cout << "EVERYTHING IS FINISHED" <<endl;
    return 0;
}

```

3.1.3.Result Collection and Analysis

The execution times obtained from the benchmark tests were stored in CSV files generated by the main program. These CSV files contain the performance data of each Quicksort pivot selection strategy for different input arrangements, including random, ascending, and descending arrays.

To visualize and analyze the collected data, a separate Python script named `graph_generator.py` was developed. The script uses the Pandas library to read and process the CSV files, while the Matplotlib library is used to generate performance graphs. The generated graphs compare the execution times of different pivot strategies and are stored in the results folder for further analysis and presentation.

graph_generator.py:

```
import pandas as pd
import matplotlib.pyplot as plt

files = [
    ("../results/random_result.csv", "Experiment 1 - Random Arrays",
    "../graphs/graph_1_random_array.png"),
    ("../results/ascending_result.csv", "Experiment 2A - Ascending
    Arrays", "../graphs/graph_2_ascending_array.png"),
    ("../results/descending_result.csv", "Experiment 2B - Descending
    Arrays", "../graphs/graph_2_descending_array.png")
]

strategies = ["First", "Last", "Middle", "Random"]

for csv_file, title, output_file in files:
    data = pd.read_csv(csv_file)
    plt.figure(figsize=(10, 12))

    for strategy in strategies:
        plt.plot(data["size"], data[strategy], label=strategy)

    plt.title(title)
    plt.xlabel("Array Size")
```

```
plt.ylabel("Average Time (microseconds)")
plt.legend()
plt.grid(True)
plt.savefig(output_file)
plt.close()

print(f"Saved {output_file}")

print("All graphs generated.")
```

4.Results

4.1.Random Array Performance

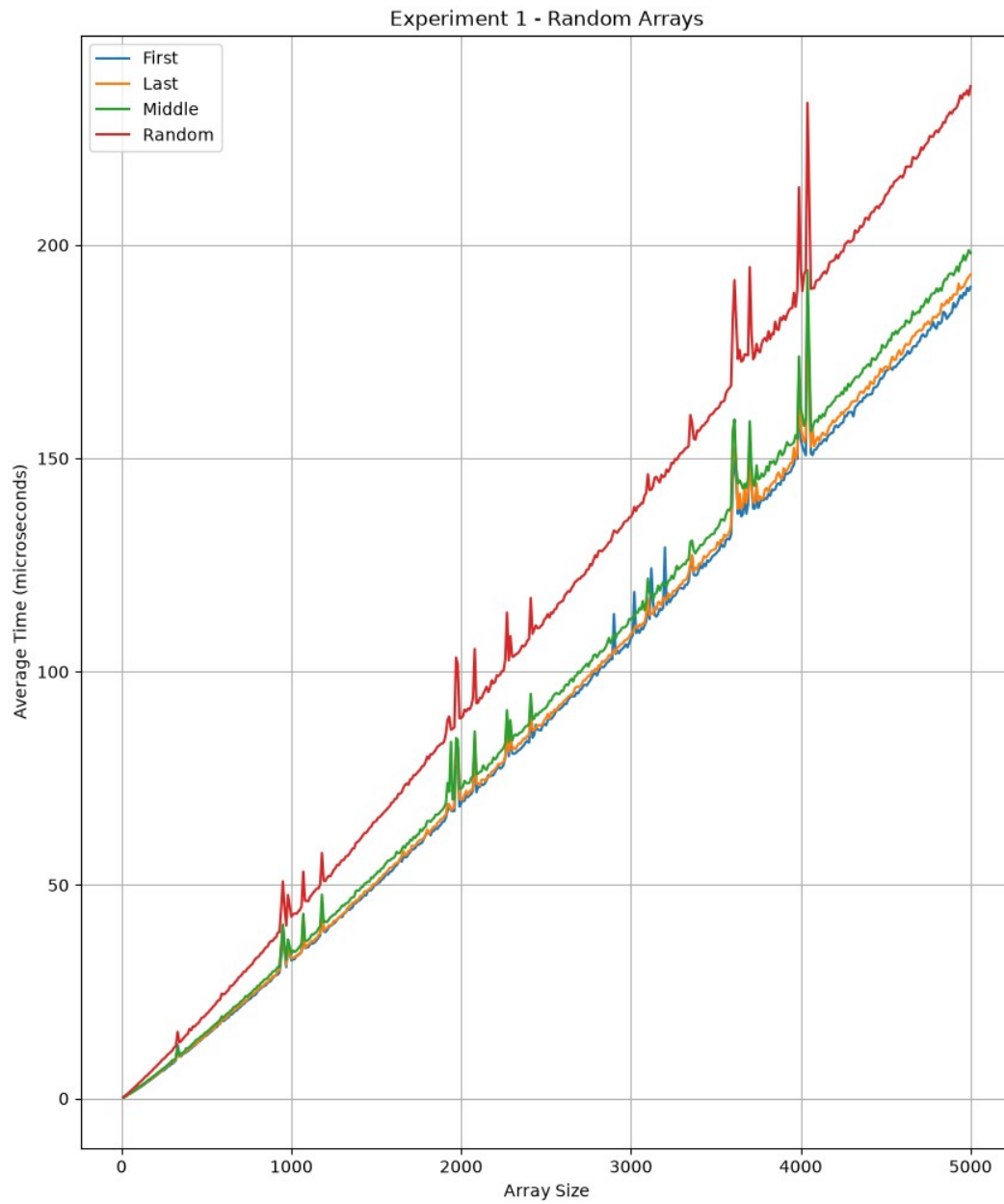


Figure 1 shows the execution time comparison of different Quicksort pivot selection strategies on randomly generated arrays. Since random input does not favor any particular pivot selection method, most strategies show relatively similar performance. However, differences can be observed as the input size increases.

4.2.Ascending Array Performance

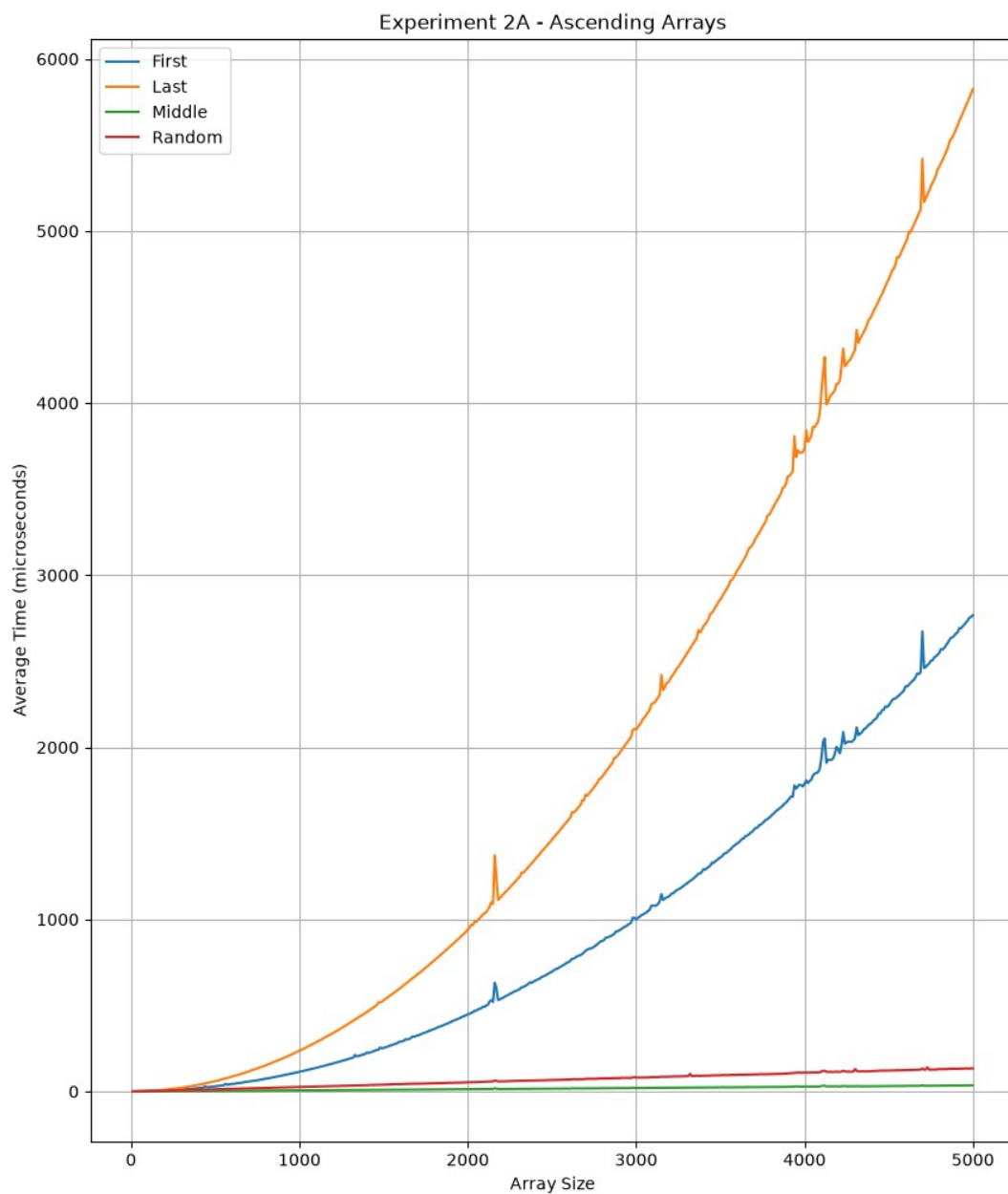


Figure 2 presents the performance of different pivot strategies on ascending arrays. The first element and last element pivot strategies show increased execution time because they produce highly unbalanced partitions on already sorted data. Middle and random pivot strategies perform better due to more balanced partitioning.

4.3.Descending Array Performance

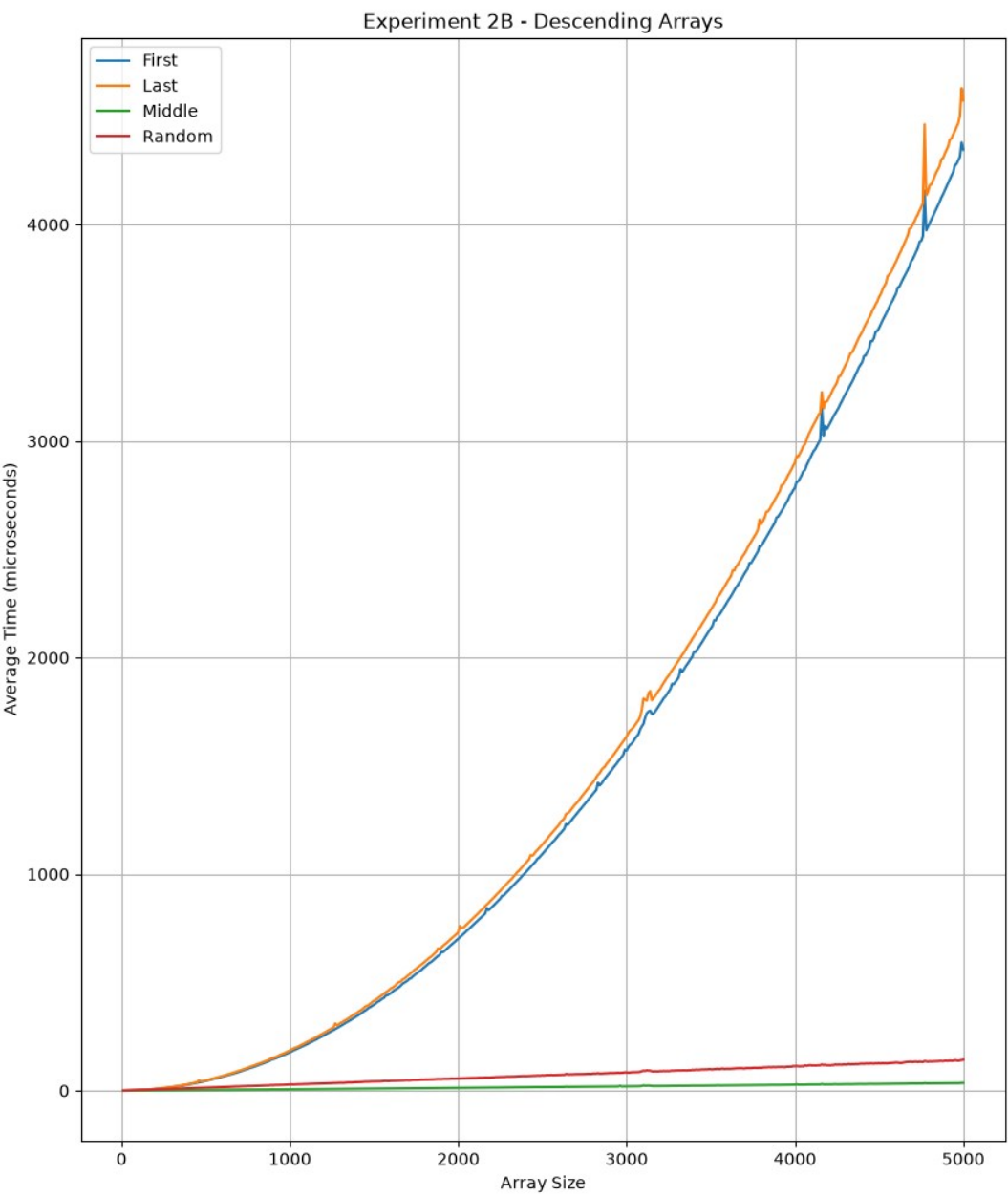


Figure 3 compares the execution time of pivot strategies on descending arrays. Similar to ascending arrays, fixed pivot selection methods can suffer from inefficient partitioning, while middle and random pivot strategies maintain more consistent performance.

5. Analysis

The experimental results show that pivot selection has a significant impact on Quicksort performance. First and last element pivot strategies perform poorly on sorted and reverse-sorted arrays because they create unbalanced partitions, leading to increased execution time. Middle and random element pivot strategies generally provide better and more consistent performance by producing more balanced partitions.

For random input arrays, all strategies show relatively similar performance. However, for ordered inputs, the difference between fixed pivot strategies and adaptive strategies becomes more noticeable. Overall, middle and random pivot selection methods are more reliable for maintaining efficient Quicksort performance across different input conditions.